# УЧЕБНИК ПО CSS

Уровень: средний

Этот учебник предназначен для тех, кто уже знаком с основами HTML и CSS, но хочет углубить знания по раскладке, селекторам, адаптивности и псевдоэлементам. Все примеры кода можно сразу проверять в браузере — достаточно сохранить их в файл style.css и подключить к HTML-странице.

Учебник охватывает восемь глав: от введения в каскад и селекторы до практических задач с разбором решений. Каждая глава содержит теорию, примеры кода и пояснения к ним, а также заканчивается краткими итогами.

---

## ОГЛАВЛЕНИЕ


- Глава 1. Введение в CSS
- Глава 2. Селекторы и каскад
- Глава 3. Бокс-модель (Box model)
- Глава 4. Flexbox
- Глава 5. CSS Grid
- Глава 6. Адаптивная вёрстка
- Глава 7. Псевдоэлементы
- Глава 8. Практические задачи и итоги


---

## ГЛАВА 1. ВВЕДЕНИЕ В CSS

### 1.1. Что такое CSS

CSS (Cascading Style Sheets, каскадные таблицы стилей) — это язык описания внешнего вида документа, написанного на языке разметки. Чаще всего CSS применяется вместе с HTML: HTML описывает структуру страницы, а CSS — её визуальное оформление: цвета, шрифты, отступы, раскладку блоков, анимации.

Без CSS веб-страница выглядела бы как обычный текстовый документ с прямыми ссылками. Именно CSS превращает набор тегов в современный интерфейс с сетками, карточками, кнопками и плавными переходами.

Понимание CSS важно не только для дизайнеров и верстальщиков, но и для фронтенд-разработчиков, потому что многие баги вёрстки невозможно исправить без чёткого знания модели бокса, каскада и алгоритмов раскладки.

## 1.2. Как подключить CSS

Есть три способа подключить стили к HTML-документу. Первый — внешняя таблица стилей через тег link в голове документа. Это самый распространённый и удобный способ, потому что стили можно переиспользовать на разных страницах и легко кэшировать.

```
&lt;link rel="stylesheet" href="style.css"&gt;
```

Второй способ — внутренние стили внутри тега style в голове HTML-документа. Удобно для прототипирования, но не подходит для больших проектов.

```
&lt;style&gt;
  body { background: #fff; }
&lt;/style&gt;
```

Третий способ — инлайновые стили через атрибут style конкретного тега. Используется редко, в основном для разовых правок или значений, которые нельзя вынести в класс.

```
&lt;p style="color: red;"&gt;Текст&lt;/p&gt;
```

На практике почти всегда выбирают первый способ: внешняя таблица стилей. Он позволяет держать структуру и оформление раздельно, что упрощает поддержку кода и командную работу.

## 1.3. Синтаксис CSS

CSS состоит из правил. Каждое правило содержит селектор и блок объявлений. Селектор указывает, к каким элементам применяется правило, а блок объявлений — пары свойство: значение.

```
селектор {
  свойство: значение;
  свойство: значение;
}
```

Пример с реальными значениями:

```
h1 {
  color: #1a1a1a;
  font-size: 32px;
  margin-bottom: 16px;
}
```

Точка с запятой после каждого объявления обязательна. Если её пропустить, браузер попытается склеить текущее свойство со следующим и проигнорирует оба. Фигурные скобки ограничивают блок объявлений, а двоеточие разделяет свойство и значение.

### 1.4. Каскад и приоритеты

Слово «каскад» в названии CSS не случайно. Когда на один элемент действует несколько правил, браузер применяет их в определённом порядке. Сначала учитывается происхождение стилей: стили автора (разработчика) перекрывают стили пользователя, а те — стили браузера по умолчанию.

Внутри одной группы стилей решает специфичность селектора. Чем «точнее» селектор указывает на элемент, тем выше его приоритет. Наконец, если специфичность одинакова, побеждает правило, объявленное позже в коде.

Это ключевая идея CSS: нельзя просто записать свойство и ожидать, что оно сработает. Нужно понимать, кто ещё претендует на этот элемент и чьё объявление победит. Подробно мы разберём это в следующей главе.

---

## ГЛАВА 2. СЕЛЕКТОРЫ И КАСКАД

### 2.1. Типы селекторов

Селектор — это шаблон, по которому браузер находит элементы в документе. Селекторы бывают простые и составные. Простые указывают на элементы по одному признаку, а составные комбинируют несколько признаков или отношений.

Основные простые селекторы:

- Универсальный: * — все элементы
- По тегу: p, div, a — элементы с указанным именем
- По классу: .card — элементы с атрибутом class="card"
- По ID: #header — элемент с id="header"
- По атрибуту: [type="text"] — элементы с указанным атрибутом

Примеры использования:

```
*      { box-sizing: border-box; }
p      { line-height: 1.5; }
.card  { padding: 16px; }
#header{ background: #fff; }
[disabled] { opacity: 0.5; }
```

## 2.2. Комбинаторы

Комбинаторы связывают простые селекторы и позволяют описать отношения между элементами. Существует четыре комбинатора:

- Пробел — потомок любого уровня: .card p
- > — прямой потомок: .card > p
- + — соседний следующий: h1 + p
- ~ — все следующие соседи: h1 ~ p

```
/* Все p внутри .card */
.card p { color: #333; }

/* Только прямые потомки */
.list &gt; li { margin-top: 4px; }

/* p, идущий сразу после h1 */
h1 + p { font-size: 18px; }

/* Все p после h1 внутри родителя */
h1 ~ p { color: #666; }
```

## 2.3. Псевдоклассы

Псевдокласс — это модификатор селектора, который уточняет состояние элемента. Записывается через двоеточие после основного селектора.

- :hover — элемент под курсором
- :focus — элемент в фокусе ввода
- :checked — отмеченный чекбокс или радиокнопка
- :first-child — первый потомок своего родителя
- :last-child — последний потомок
- :nth-child(n) — n-й потомок по формуле
- :not(s) — все, кроме подходящих под селектор s

```
a:hover { color: #0066cc; }
input:focus { border-color: #0066cc; }
li:first-child { font-weight: bold; }
li:nth-child(odd) { background: #f5f5f5; }
li:not(:first-child) { margin-top: 8px; }
```

Псевдокласс :nth-child особенно полезен для зеброчной подсветки таблиц и списков. В скобках можно указать формулу вида an+b, например 2n+1 для нечётных позиций или 3n для каждого третьего элемента.

## 2.4. Формулы в :nth-child

Псевдокласс :nth-child принимает не только конкретные числа, но и формулы вида an+b. Здесь a — шаг, n — счётчик от нуля, b — смещение. Это мощный инструмент, который заменяет десятки ручных классов.

Самые распространённые формулы:

- odd или 2n+1 — нечётные позиции (1, 3, 5, ...)
- even или 2n — чётные позиции (2, 4, 6, ...)
- 3n — каждый третий (3, 6, 9, ...)
- -n+3 — первые три элемента (3, 2, 1)
- n+4 — все, начиная с четвёртого

```
/* Выделить первые три элемента */
li:nth-child(-n+3) { color: red; }

/* Каждый четвёртый, начиная со 2-го */
tr:nth-child(4n+2) { background: #eee; }
```

Особенно интересно работает селектор :nth-of-type, который считает только элементы указанного типа. Это удобно, когда среди потомков есть смесь разных тегов, и нужно считать только p или только article.

```
/* Каждый второй &lt;p&gt; внутри .post */
.post p:nth-of-type(even) {
  background: #f9f9f9;
}
```

## 2.5. Специфичность

Специфичность — это числовая оценка селектора, которая определяет его приоритет в каскаде. Она записывается тремя числами в формате (a, b, c):

- a — количество ID в селекторе (#header)

- b — классы, атрибуты и псевдоклассы
- c — теги и псевдоэлементы

Универсальный селектор * и комбинаторы (пробел, >, +, ~) не добавляют специфичности. Чем больше каждое из чисел слева направо, тем выше приоритет. Сравнивают по принципу лексикографического порядка: (1, 0, 0) сильнее (0, 99, 99).

Примеры специфичности:

- * → (0,0,0)
- p → (0,0,1)
- .card → (0,1,0)
- #header → (1,0,0)
- .card p → (0,1,1)
- #header .title → (1,1,0)
- div.card > a:hover → (0,2,2)

## 2.6. Слои @layer

Современный CSS предлагает механизм @layer — именованные слои стилей, которые управляются явно, а не только через специфичность. Слои задаются в начале файла, и порядок их объявления определяет приоритет: последний слой побеждает.

```
@layer reset, base, components, utils;

@layer reset {
  * { box-sizing: border-box; }
  body { margin: 0; }
}

@layer base {
  body { font-size: 16px; }
}

@layer utils {
  .text-center { text-align: center; }
}
```

Стиль внутри любого слоя всегда приоритетнее стиля вне слоёв, а внутри слоёв работает обычная специфичность. Это решает извечную проблему подключения сторонних библиотек: поместив reset в самый нижний слой, можно гарантировать, что собственные стили всегда перекроют стили библиотеки.

## 2.7. !important: когда не нужен

Объявление !important повышает приоритет свойства выше любой специфичности. Это крайняя мера, к которой прибегают, когда никакой селектор не помогает перекрыть уже существующее значение.

```
.btn { color: red !important; }
```

На практике !important почти всегда признак проблемы в архитектуре стилей. Вместо него лучше повысить специфичность селектора или переупорядочить правила. Если !important появляется часто, скорее всего, разработчик борется сам с собой: стили плохо структурированы, и каждый новый патч вынужден «перекрикивать» предыдущие. Лечится это разбором каскада и наведением порядка в именах классов.

## 2.8. Наследование

Некоторые свойства CSS автоматически наследуются потомками. К наследуемым относятся: color, font-family, font-size, line-height, text-align и другие свойства, связанные с текстом и шрифтом. Ненаследуемые — это в основном «коробочные» свойства: margin, padding, border, background.

Управлять наследованием можно ключевыми словами inherit (явно наследовать), initial (значение по умолчанию) и unset (наследовать, если свойство наследуется, иначе сбросить).

```
.btn {
  color: inherit;  /* наследуем цвет */
  border: initial; /* сброс в умолчание */
}
```

С ключевым словом currentColor можно сослаться на текущее значение свойства color. Это удобно, когда нужно, чтобы другое свойство (например, цвет рамки) всегда соответствовало тексту.

```
.icon {
  color: #0066cc;
  border: 1px solid currentColor;
  fill: currentColor;
}
```

# ГЛАВА 3. БОКС-МОДЕЛЬ (BOX MODEL)

## 3.1. Что такое бокс-модель

Каждый элемент на странице браузер представляет как прямоугольную коробку (бокс). Эта коробка состоит из четырёх областей: содержимое (content), внутренний отступ (padding), рамка (border) и внешний отступ (margin). Понимание того, как эти области складываются, — фундамент всей вёрстки на CSS.

Область содержимого — это место, где находится текст или вложенные элементы. Padding отделяет содержимое от рамки и принадлежит самому элементу, в том числе участвует в кликабельной области и фоновом цвете. Border — рамка вокруг padding. Margin — внешний отступ, который разделяет текущий элемент от соседних.

Важно: вертикальные margin соседних блочных элементов схлопываются. Если у первого margin-bottom: 20px, а у второго margin-top: 30px, между ними будет не 50, а 30 пикселей — побеждает большее значение. Это поведение называют схлопыванием margin.

## 3.2. content-box и border-box

По умолчанию используется модель content-box: свойства width и height задают размер только содержимого, а padding и border добавляются сверху. Если задать width: 200px и padding: 20px, итоговая ширина получится 240px плюс толщина рамки.

Модель border-box удобнее: width и height включают в себя padding и border. Задал 200px — получил ровно 200px, независимо от отступов и рамки. Подавляющее большинство современных проектов переключается на border-box глобально следующим правилом:

```
*, *::before, *::after {
  box-sizing: border-box;
}
```

Эту строку обычно помещают в самое начало файла стилей. Она меняет поведение всех элементов, но именно border-box соответствует интуитивному ожиданию: заданная ширина равна видимой ширине.

## 3.3. Единицы измерения

CSS поддерживает много единиц измерения, но на практике чаще всего используются следующие:

- px — пиксели, абсолютная единица

- em — относительно шрифта родителя
- rem — относительно шрифта корня (html)
- % — относительно размера родителя
- vw, vh — 1% от ширины/высоты окна
- fr — доля свободного места в Grid

Различие em и rem критично. Единица em зависит от размера шрифта текущего элемента, который сам может зависеть от родителя. Это создаёт эффект компаундинга: вложенные элементы масштабируются друг относительно друга и предсказать итоговый размер сложно.

```
.card           { font-size: 18px; }
.card .title    { font-size: 1.2em; /* 21.6px */ }
.card .title .x { font-size: 1.2em; /* 25.92px */ }
```

rem (root em) всегда считается от шрифта корневого элемента html. В большинстве браузеров это 16px. Использование rem делает размеры предсказуемыми и легко масштабируемыми: достаточно поменять font-size у html, и весь интерфейс адаптируется.

```
html { font-size: 16px; }
h1   { font-size: 2rem;   /* 32px */ }
p    { font-size: 1rem;   /* 16px */ }
```

## 3.4. Цвета

Цвета в CSS можно задавать несколькими способами. HEX-запись — это шесть шестнадцатеричных цифр, по две на канал RGB. RGB и RGBA — функция с числами от 0 до 255. HSL задаёт цвет через тон (hue), насыщенность (saturation) и светлоту (lightness) — это самый удобный формат для подбора оттенков.

```
/* Эквивалентные записи синего цвета */
color: #0066cc;
color: rgb(0, 102, 204);
color: rgba(0, 102, 204, 0.5);
color: hsl(210, 100%, 40%);
color: hsla(210, 100%, 40%, 0.5);
```

Современные браузеры поддерживают восьмизначный HEX с альфа-каналом: #0066cc80 — последний байт задаёт прозрачность от 00 (полная прозрачность) до FF (полная непрозрачность). Также есть функция color-mix() для смешивания двух цветов прямо в CSS, без препроцессоров.

## 3.5. Шрифты и текст

Семейство шрифта задаётся свойством font-family. Обычно указывают несколько шрифтов через запятую, заканчивая универсальным семейством — браузер выберет первый доступный.

```
body {
  font-family: "Segoe UI", Roboto,
               Helvetica, Arial,
               sans-serif;
}
```

Межстрочный интервал line-height удобнее задавать без единиц: тогда он интерпретируется как множитель к текущему размеру шрифта. Значение 1.5 — самый распространённый выбор для читаемого основного текста.

```
body {
  font-size: 16px;
  line-height: 1.5;  /* 24px */
}
```

## 3.6. Схлопывание margin

Вертикальные margin соседних блочных элементов схлопываются: между ними остаётся большее из двух значений, а не их сумма. Это поведение часто удивляет новичков, но оно удобно для текстовых абзацев: соседние <p> разделяются одним промежутком, а не двойным.

Схлопывание работает только по вертикали между блочными элементами. Flex- и grid-контейнеры отключают схлопывание для своих потомков, поэтому во Flexbox и Grid лучше использовать gap. Также margin не схлопывается, если у элемента есть padding, border, или если задано overflow: hidden.

Чтобы избежать схлопывания, достаточно превратить родителя в flex- или grid-контейнер, либо добавить ему padding-top: 1px. Это частый приём в карточках и панелях, где важна предсказуемость отступов.

## 3.7. Дисплей и контексты форматирования

Свойство display определяет, как элемент ведёт себя в потоке документа. Основные значения: block, inline, inline-block, flex, grid, none. От значения display зависит, какие свойства применимы и как элемент взаимодействует с соседями.

- block — занимает всю ширину, переносит строку
- inline — обтекает текстом, размеры не работают
- inline-block — обтекает, но размеры можно задавать

- flex/grid — становятся контейнерами для раскладки
- none — элемент удаляется из потока и не виден

Различие inline и inline-block критично для кнопок и иконок. Inline-элемент нельзя точно позиционировать по размеру, а inline-block — можно. Но inline-block имеет особенность: между соседними inline-block элементами образуется пробел в 4 пикселя из-за пробельных символов в HTML. Это лечится либо Flexbox, либо приёмом font-size: 0 на родителе.

---

# ГЛАВА 4. FLEXBOX

## 4.1. Что такое Flexbox

Flexbox (Flexible Box Layout) — это модуль CSS для одномерной раскладки. «Одномерной» означает, что элементы выстраиваются вдоль одной оси: либо в строку, либо в столбец. Flexbox идеально подходит для горизонтальных меню, карточек в ряд, панелей инструментов и вообще любых блоков, которые нужно выровнять по одной линии.

Чтобы включить Flexbox, нужно задать родителю свойство display: flex. После этого прямые потомки становятся flex-элементами, а у родителя появляются новые свойства для управления раскладкой.

```
.menu { display: flex; }
.menu a { padding: 8px 16px; }
```

## 4.2. Главная и поперечная ось

Во Flexbox есть две оси. Главная ось (main axis) задаётся свойством flex-direction и определяет направление, в котором раскладываются элементы. Поперечная ось (cross axis) перпендикулярна главной.

- row — слева направо (по умолчанию)
- row-reverse — справа налево
- column — сверху вниз
- column-reverse — снизу вверх

Это различие принципиально: свойства justify-content выравнивают по главной оси, а align-items — по поперечной. Перепутать их — самая частая ошибка начинающих.

## 4.3. justify-content

Свойство justify-content управляет распределением свободного пространства вдоль главной оси. Применяется к flex-контейнеру.

- flex-start — элементы в начале (по умолчанию)
- flex-end — элементы в конце
- center — элементы по центру
- space-between — равные промежутки, по краям 0
- space-around — половина промежутка по краям
- space-evenly — одинаковые промежутки везде

```
.menu {
  display: flex;
  justify-content: space-between;
}
```

## 4.4. align-items и align-self

align-items задаёт выравнивание элементов по поперечной оси. Применяется к контейнеру, влияет сразу на все flex-дети.

- stretch — растянуть (по умолчанию)
- flex-start — прижать к началу поперечной оси
- flex-end — прижать к концу
- center — по центру
- baseline — по базовой линии текста

align-self переопределяет align-items для конкретного элемента. Удобно, когда нужно выделить одну кнопку из ряда.

```
.row { display: flex; align-items: center; }
.row .tall { align-self: stretch; }
```

## 4.5. flex-wrap и gap

По умолчанию flex-элементы пытаются уместиться в одну линию, даже если для этого их приходится сжимать. Свойство flex-wrap: wrap разрешает перенос на новую строку, когда места не хватает.

```
.cards {
  display: flex;
  flex-wrap: wrap;
  gap: 16px;
}
```

Свойство gap задаёт расстояние между элементами и работает во Flexbox, Grid и многоколоночной раскладке. Это удобнее, чем прописывать margin каждому элементу и потом убирать его у первого или последнего.

## 4.6. flex-grow, flex-shrink, flex-basis

Это три свойства, управляющие распределением свободного места между flex-элементами. Обычно их задают вместе через сокращение flex.

- flex-grow — во сколько раз элемент забирает место
- flex-shrink — насколько элемент готов сжиматься
- flex-basis — стартовый размер до распределения

```
.sidebar { flex: 0 0 240px; }  /* фикс. */
.content { flex: 1 1 0; }       /* растяг. */
```

Запись flex: 1 1 0 означает: элемент забирает всю свободную область (grow=1), готов сжиматься (shrink=1), стартовый размер нулевой (basis=0). Это распространённый паттерн для резинового контента рядом с фиксированной боковой колонкой.

## 4.7. Типичные паттерны Flexbox

Горизонтальное центрирование блока — классическая задача, которая до Flexbox требовала таблиц или трюков с margin: 0 auto. Теперь это две строки:

```
.center {
  display: flex;
  justify-content: center;
  align-items: center;
  min-height: 100vh;
}
```

Боковая колонка фиксированной ширины и резиновый основной контент:

```
.layout {
  display: flex;
  gap: 24px;
}
.sidebar { flex: 0 0 240px; }
.main    { flex: 1; }
```

### 4.8. Свойство order

Свойство order меняет визуальный порядок flex-элементов без изменения HTML. По умолчанию order: 0, и элементы идут в порядке объявления. Чем меньше order, тем раньше элемент.

```
.first  { order: 2; }  /* будет вторым */
.second { order: 1; }  /* будет первым */
.third  { order: 3; }  /* будет третьим */
```

Важно: order меняет только визуальный порядок, но не порядок в DOM. Это значит, что скринридеры и навигация с клавиатуры по-прежнему идут по HTML-порядку. Рассогласование визуального и DOM-порядка может запутать пользователя, поэтому order применяют осторожно и только для декоративных целей.

### 4.9. Типичные ошибки Flexbox

Самая частая ошибка — путать justify-content и align-items. Запомнить просто: justify работает по главной оси, align — по поперечной. При flex-direction: row главная ось горизонтальная, поэтому justify-content: center центрирует по горизонтали. При flex-direction: column главная ось становится вертикальной, и justify-content: center центрирует уже по вертикали.

Вторая типичная ошибка — задавать width flex-элементам вместе с flex-grow. Свойство flex-basis в сокращении flex задаёт стартовый размер, и если указать ещё и width, поведение может оказаться неожиданным. Старайтесь использовать либо flex-basis, либо width, но не оба сразу.

Третья ошибка — забывать про flex-shrink. По умолчанию flex-shrink: 1, и элементы сжимаются при нехватке места. Если нужно сохранить минимальный размер, задайте flex-shrink: 0 или min-width: ...

---

# ГЛАВА 5. CSS GRID

## 5.1. Что такое Grid

CSS Grid Layout — это модуль двумерной раскладки. В отличие от Flexbox, который работает с одной осью, Grid одновременно управляет и строками, и столбцами. Это идеальный инструмент для сложных страничных макетов: сетки галерей, дашбордов, журнальных вёрсток с несколькими колонками и областями.

Включается Grid свойством display: grid. Размеры дорожек (колонок и строк) задаются свойствами grid-template-columns и grid-template-rows.

```
.grid {
  display: grid;
  grid-template-columns: 200px 1fr;
  grid-template-rows: auto 1fr auto;
  gap: 16px;
  min-height: 100vh;
}
```

## 5.2. Единицы fr и функция repeat

Единица fr (fraction) обозначает долю свободного места в Grid-контейнере. grid-template-columns: 1fr 1fr 1fr создаёт три равные колонки, а 2fr 1fr — две колонки в пропорции 2:1.

Функция repeat() позволяет не писать повторяющиеся значения вручную. Запись repeat(3, 1fr) эквивалентна 1fr 1fr 1fr, а repeat(auto-fill, minmax(200px, 1fr)) автоматически создаёт столько колонок по 200px минимум, сколько поместится в контейнере, и растягивает их до заполнения.

```
.gallery {
  display: grid;
  grid-template-columns:
    repeat(auto-fill, minmax(200px, 1fr));
  gap: 16px;
}
```

Это паттерн для адаптивных галерей и карточек без единого медиазапроса: число колонок само подстраивается под ширину контейнера.

## 5.3. minmax и auto-fit

Функция minmax() задаёт диапазон размеров: минимальный и максимальный. Колонка будет не меньше минимума и не больше максимума. В комбинации с auto-fit (вместо auto-fill) даёт эффект «растягивающихся карточек»: если элементов мало, они занимают всё доступное место; если много — переносятся на новую строку.

```
.cards {
  display: grid;
  grid-template-columns:
    repeat(auto-fit, minmax(240px, 1fr));
  gap: 16px;
}
```

## 5.4. grid-area и именованные области

Grid позволяет задавать имена областям и раскладывать элементы по этим именам. Это самый наглядный способ описать макет страницы: видно, что куда помещается, без вычислений колонок и строк.

```
.page {
  display: grid;
  grid-template-areas:
    "header header"
    "side   main"
    "footer footer";
  grid-template-columns: 200px 1fr;
  grid-template-rows: auto 1fr auto;
  min-height: 100vh;
}
.header  { grid-area: header; }
.sidebar { grid-area: side;   }
.main    { grid-area: main;   }
.footer  { grid-area: footer; }
```

Имена областей должны образовывать прямоугольники — нельзя задать «дырявую» или L-образную область. Каждое имя в шаблоне должно встречаться непрерывным блоком. Точка в шаблоне обозначает пустую ячейку.

## 5.5. Явное размещение элементов

Помимо именованных областей, элементы можно размещать явно через grid-column и grid-row, указывая номера линий или диапазоны через слэш.

```
.item-a {
  grid-column: 1 / 3;  /* с 1-й по 3-ю линию */
  grid-row: 2 / 3;
}
.item-b {
  grid-column: 1;
  grid-row: 1 / span 2;  /* 2 строки */
}
```

Ключевое слово span указывает, сколько дорожек займёт элемент, а не до какой линии дойти. Удобно, когда точный номер линии неизвестен.

## 5.6. Flexbox или Grid?

Распространённый вопрос новичков: что выбрать — Flexbox или Grid? Краткий ответ: Flexbox для одномерной раскладки (одна линия элементов), Grid для двумерной (сетка). На практике они часто комбинируются: внешний каркас страницы — на Grid, а внутри блоков меню и кнопок — Flexbox.

Также Grid справляется с задачами, которые Flexbox не умеет: точное выравнивание по двум осям одновременно, наложение элементов друг на друга через один и тот же grid-area, управление порядком без изменения HTML.

## 5.7. Выравнивание в Grid

В Grid есть отдельные свойства для выравнивания элементов: justify-items (по горизонтали), align-items (по вертикали) и их сокращение place-items. Аналогично justify-content и align-content выравнивают всю сетку внутри контейнера, если она меньше доступного места.

```
.grid {
  display: grid;
  grid-template-columns: 1fr 1fr;
  justify-items: center;
  align-items: stretch;
  /* или одной строкой: */
  place-items: center stretch;
}
```

Отдельный элемент можно выровнять через justify-self и align-self, которые переопределяют значения контейнера. Это полезно для особых карточек, которые должны занимать всю высоту строки или, наоборот, выравниваться по верху.

### 5.8. dense packing

По умолчанию Grid кладёт элементы в порядке объявления: каждый следующий ищет первое свободное место в потоке. Свойство grid-auto-flow: dense разрешает заполнять пустые ячейки, оставшиеся после крупных элементов. Это полезно для галерей с элементами разной высоты: картинка, занявшая две строки, оставит дыру, которую dense заполнит следующим подходящим по размеру.

```
.gallery {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  grid-auto-flow: row dense;
  gap: 12px;
}
.tall { grid-row: span 2; }
```

Осторожно: dense может переупорядочить элементы неочевидным для пользователя образом. Это плохо для табличных данных и списков с логическим порядком. Хорошо — для декоративных галерей, где порядок не принципиален.

---

# ГЛАВА 6. АДАПТИВНАЯ ВЁСТКА

### 6.1. Мобильный подход (mobile-first)

Адаптивная вёрстка — это подход, при котором один и тот же HTML отображается по-разному в зависимости от ширины экрана. Ключевая идея mobile-first: сначала пишут стили для мобильных устройств (по умолчанию), а затем через медиазапросы расширяют раскладку для больших экранов.

Этот подход хорош тем, что мобильные стили обычно проще: одна колонка, вертикальное меню, минимум декора. По мере роста экрана добавляются колонки, появляются боковые панели, перестраивается навигация. Базовые стили работают везде, а медиазапросы только дополняют их.

### 6.2. Медиазапросы

Медиазапрос — это конструкция @media, которая применяет стили только при определённых условиях. Чаще всего проверяют min-width и max-width.

```
/* Базовые мобильные стили */
.grid { display: block; }

/* С 768px и шире: две колонки */
@media (min-width: 768px) {
  .grid {
    display: grid;
    grid-template-columns: 1fr 1fr;
  }
}

/* С 1200px и шире: три колонки */
@media (min-width: 1200px) {
  .grid {
    grid-template-columns: repeat(3, 1fr);
  }
}
```

## 6.3. Точки перелома (breakpoints)

Точка перелома — это значение ширины, при котором раскладка меняется. Их выбирают не по размерам конкретных устройств (iPhone 14, iPad Air и т.д.), а по содержимому: когда текущая раскладка начинает выглядеть плохо, добавляют новую точку. Это правило называется «breakpoints by content, not by device».

Тем не менее, есть устоявшиеся ориентиры, от которых можно отталкиваться:

- до 640px — мобильные телефоны
- 640–1024px — планшеты
- 1024–1440px — ноутбуки
- свыше 1440px — большие экраны

Не стоит плодить десятки точек перелома. Обычно достаточно 2–3 значений, чтобы покрыть основные сценарии. Чем меньше точек, тем проще поддерживать код.

## 6.4. Относительные единицы и clamp()

Адаптивность не ограничивается медиазапросами. Часто достаточно пользоваться относительными единицами и функциями, которые сами подстраивают размер под ширину окна.

Функция clamp(min, preferred, max) задаёт значение, которое не меньше min, не больше max, а в обычном случае равно preferred. Это идеальный инструмент для адаптивной типографики:

```
h1 {
  font-size: clamp(24px, 5vw, 48px);
}
p {
  font-size: clamp(16px, 2vw, 18px);
  line-height: 1.5;
}
```

В этом примере заголовок будет расти вместе с окном просмотра, но никогда не станет меньше 24 или больше 48 пикселей. Это полностью заменяет несколько медиазапросов для типографики.

## 6.5. Адаптивные изображения

Адаптивность касается не только раскладки, но и медиа. Картинка с фиксированным width=1200px плохо смотрится на мобильном экране. Базовое правило: задавать изображениям max-width: 100%, чтобы они никогда не вылазили за пределы родителя.

```
img {
  max-width: 100%;
  height: auto;
  display: block;
}
```

Для retina-экранов и разных плотностей пикселей используют атрибут srcset тега img или элемент picture. Браузер сам выбирает подходящий файл в зависимости от размера экрана и плотности пикселей.

```
&lt;img src="small.jpg"
  srcset="small.jpg 480w,
          medium.jpg 1024w,
          large.jpg 1920w"
  sizes="(min-width: 768px) 50vw,
          100vw"
  alt="Описание"&gt;
```

Элемент picture даёт ещё больше контроля: можно переключаться между разными форматами (WebP, AVIF) и даже разными обрезками изображения для разных ориентаций экрана. Это особенно важно для художественной фотографии, где горизонтальный и вертикальный кадр кардинально отличаются.

### 6.6. Контейнерные запросы

Контейнерные запросы (container queries) — это относительно новое дополнение к CSS, которое позволяет применять стили в зависимости не от ширины окна, а от ширины родительского контейнера. Это особенно полезно для компонентов: карточка может выглядеть по-разному в узкой боковой колонке и в широком основном блоке, реагируя на размер своего родителя, а не окна.

```
.sidebar { container-type: inline-size; }

@container (min-width: 320px) {
  .card { display: grid;
    grid-template-columns: 1fr 1fr; }
}
```

---

# ГЛАВА 7. ПСЕВДОЭЛЕМЕНТЫ

### 7.1. Что такое псевдоэлементы

Псевдоэлемент — это виртуальный элемент, который создаётся CSS и не существует в HTML. Он позволяет добавлять декоративное содержимое или стилизовать части текста без дополнительных тегов. Записывается через двойное двоеточие, чтобы отличать от псевдоклассов, которые используют одинарное.

Самые полезные псевдоэлементы:

- ::before — контент перед элементом
- ::after — контент после элементом
- ::first-letter — первая буква блока
- ::first-line — первая строка блока
- ::selection — выделенный пользователем текст
- ::placeholder — placeholder в полях ввода

### 7.2. ::before и ::after

Псевдоэлементы ::before и ::after создают виртуальные элементы внутри целевого. ::before помещается перед основным содержимым, ::after — после. Чтобы псевдоэлемент что-то отображал, обязательно нужно задать свойство content, даже если это пустая строка.

```
.btn::before {
  content: "\2192";  /* стрелка → */
  margin-right: 8px;
}

.required::after {
  content: "*";
  color: red;
  margin-left: 4px;
}
```

Часто ::before и ::after используют для декоративных целей: иконок, подчёркиваний, наложений, углов. Главное правило: семантически важное содержимое не должно быть в псевдоэлементах, потому что оно не попадает в DOM, не индексируется поисковиками и недоступно скринридерам.

## 7.3. ::first-letter и ::first-line

Псевдоэлемент ::first-letter стилизует первую букву блочного элемента. Используется для типографского эффекта «буквицы» — увеличенной заглавной буквы в начале абзаца.

```
article p::first-letter {
  font-size: 3em;
  font-weight: bold;
  float: left;
  margin-right: 8px;
  line-height: 1;
}
```

Псевдоэлемент ::first-line стилизует первую строку текста. Полезен для статей и записей в блогах:

```
article p::first-line {
  font-variant: small-caps;
  letter-spacing: 0.05em;
}
```

## 7.4. ::selection

Псевдоэлемент ::selection задаёт стиль выделенного пользователем текста. Поддерживаются не все свойства, в основном color, background-color и text-shadow.

```
::selection {
  background: #0066cc;
  color: #fff;
}
```

## 7.5. Счётчики CSS

Счётчики CSS — это механизм автоматической нумерации. Состоит из трёх свойств: counter-reset (создаёт счётчик), counter-increment (увеличивает), и функция counter() или counters() (выводит значение через content).

```
ol { counter-reset: item; }
li { counter-increment: item; }
li::before {
  content: counter(item) ". ";
  color: #0066cc;
  font-weight: bold;
}
```

Функция counters() с разделителем позволяет строить многоуровневую нумерацию: 1, 1.1, 1.2, 2, 2.1 и так далее. Это удобно для оглавлений и иерархических списков.

```
ol { counter-reset: section; }
li {
  counter-increment: section;
}
li::before {
  content: counters(section, ".") " ";
}
```

## 7.6. ::placeholder и ::marker

Псевдоэлемент ::placeholder стилизует текст-подсказку в полях ввода. Поддержка ограничена: можно менять цвет, шрифт, но не все свойства работают одинаково во всех браузерах.

```
input::placeholder {
  color: #999;
  font-style: italic;
}
```

Псевдоэлемент ::marker стилизует маркер списка: точку в ul, номер в ol. Можно менять цвет, шрифт, содержимое.

```
li::marker {
  color: #0066cc;
  font-weight: bold;
}

summary::marker { content: "\25B6 "; }
```

### 7.7. Практические приёмы

Псевдоэлементы открывают много приёмов, которые без них были бы невозможны или требовали бы лишних тегов. Вот несколько полезных паттернов.

Во-первых, подчёркивание ссылок другим цветом, чем сам текст. Контент ссылки — одного цвета, а ::after с position: absolute создаёт линию другого цвета под текстом.

Во-вторых, иконки-стрелки на кнопках. Юникод-символ в content и небольшой margin заменяют десятки span в HTML.

В-третьих, декоративные кавычки для цитат. ::before и ::after с content: "\201C" и "\201D" автоматически оборачивают текст в типографские кавычки, не трогая разметку.

```
blockquote::before {
  content: "\201C";
  font-size: 2em;
  color: #999;
  line-height: 0;
  vertical-align: -0.3em;
}
blockquote::after {
  content: "\201D";
  font-size: 2em;
  color: #999;
  line-height: 0;
  vertical-align: -0.3em;
}
```

---

# ГЛАВА 8. ПРАКТИЧЕСКИЕ ЗАДАЧИ И ИТОГИ

## 8.1. Задача 1: карточка товара

Нарисуйте карточку товара: изображение сверху, заголовок, цена и кнопка «в корзину». Ширина карточки — 280px, содержимое центрировано, между элементами — 12px вертикального отступа. Решение:

```
.card {
  width: 280px;
  padding: 16px;
  display: flex;
  flex-direction: column;
  gap: 12px;
  text-align: center;
  border: 1px solid #e0e0e0;
  border-radius: 8px;
}
.card img { width: 100%;
  border-radius: 4px; }
.card .price { font-weight: bold;
  color: #0066cc; }
```

## 8.2. Задача 2: шапка с навигацией

Сделайте шапку: логотип слева, меню справа, всё по центру по вертикали, высота шапки 64px. Решение:

```
.header {
  display: flex;
  align-items: center;
  justify-content: space-between;
  height: 64px;
  padding: 0 24px;
  background: #fff;
  border-bottom: 1px solid #eee;
}
.nav { display: flex; gap: 24px; }
```

## 8.3. Задача 3: сетка галереи

Галерея из картинок: 3 колонки на десктопе, 2 на планшете, 1 на мобильном. Между картинками — 16px. Решение через Grid и медиазапросы:

```
.gallery {
  display: grid;
  gap: 16px;
  grid-template-columns: 1fr;
}
@media (min-width: 640px) {
  .gallery {
    grid-template-columns: 1fr 1fr;
  }
}
@media (min-width: 1024px) {
  .gallery {
    grid-template-columns:
      repeat(3, 1fr);
  }
}
```

## 8.4. Задача 4: кнопка с иконкой

Кнопка с декоративной стрелкой через ::before. Стрелка — юникод-символ, отступ справа 8px. Решение:

```
.btn {
  display: inline-flex;
  align-items: center;
  padding: 8px 16px;
  background: #0066cc;
  color: #fff;
  border: none;
  border-radius: 4px;
  cursor: pointer;
}
.btn::before {
  content: "\2192";
  margin-right: 8px;
}
```

## 8.5. Задача 5: адаптивная типографика

Заголовок страницы должен быть 32px на десктопе, 24px на мобильных, и плавно меняться между этими значениями. Решение через clamp():

```
.page-title {
  font-size: clamp(24px, 4vw, 32px);
  line-height: 1.2;
  margin-bottom: 16px;
}
```

## 8.6. Итоги по главам

Глава 1. CSS — это язык описания внешнего вида HTML. Состоит из правил вида селектор { свойства }. Каскад определяет, какое из нескольких правил победит.

Глава 2. Селекторы находят элементы по тегу, классу, ID, атрибуту, состоянию и положению в дереве. Специфичность считается как (a, b, c) и решает, кто победит при конфликте. !important — крайняя мера, которой лучше избегать.

Глава 3. Каждый элемент — это бокс из content, padding, border, margin. Модель border-box удобнее content-box, её включают глобально через *. Единицы rem предсказуемее em, потому что считаются от корня.

Глава 4. Flexbox — для одномерной раскладки. Главная ось задаётся flex-direction, justify-content выравнивает по главной, align-items — по поперечной. flex: 1 1 0 — паттерн для резинового элемента.

Глава 5. Grid — для двумерной раскладки. Колонки и строки задаются через grid-template-*. Единица fr — доля свободного места, repeat() и minmax() — для адаптивных сеток без медиазапросов.

Глава 6. Адаптивность начинается с mobile-first и относительных единиц. Медиазапросы @media (min-width: ...) расширяют стили для больших экранов. clamp() заменяет несколько медиазапросов для типографики.

Глава 7. Псевдоэлементы ::before и ::after добавляют декоративный контент через свойство content. ::first-letter и ::first-line стилизуют части текста. Счётчики CSS автоматизируют нумерацию через counter-reset и counter-increment.

---

## Что читать дальше

После освоения материала этого учебника логично двигаться в трёх направлениях. Во-первых, анимации и переходы: transition, transform, @keyframes. Во-вторых, препроцессоры и архитектура: Sass, CSS-модули, методологии BEM и Atomic. В-третьих, современные возможности CSS: кастомные свойства (переменные), функции color-mix(), nest-правила, слой @layer.

Главный совет: не пытайтесь заучить все свойства и значения. CSS — это практика. Берите настоящие задачи — свой блог, расписание на неделю, витрину для портфолио — и верстайте. С каждой новой страницей приходит интуитивное понимание, что выбрать: flex или grid, rem или px, медиазапрос или clamp. Опыт складывается из десятков сделанных и переделанных макетов, а не из прочитанных справочников.

---